



POLITECNICO
MILANO 1863

Version control

ACTAM 2026-2027

Andrea Rizzini

September 18, 2026

- Short introduction (~20 min)
- Short GitHub demo (~10 min)
- **Hands-on:** Setup and play with first GitHub account (~30 min)
- Break (~10 min)
- Short demo using Git (~10 min)
- **Hands-on:** Solo repo (~20 min)
- **Hands-on:** Collab repo (∞)

- To do many things at once (non-linear development)
- To go back in history
- To undo mistakes
- To keep control

- Folders, documents, codebases. . . a song?
- Markdown!

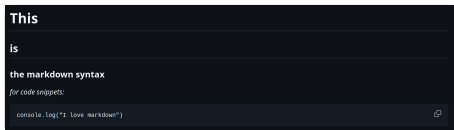
Rule of thumb

Git shines on **plain text** (code, Markdown, CSV, config files): it can compare two versions line by line. Binary files are tracked too, but only as “old file / new file”: no meaningful diff, and the repository grows quickly.

This is *the* Markdown syntax

4/22

```
# This
## is
### the markdown syntax
_for code snippets:
'''
console.log("I love markdown")
'''
```



rendered output

Explore more at:

- <https://markdownlivepreview.com/>
- <https://www.markdownguide.org/cheat-sheet/>



Linus Torvalds

torvalds

Follow

216k followers · 0 following

Linux Foundation
Portland, OR

Achievements






Popular repositories

linux

Linux kernel source tree

● C ☆ 179k 🍴 53.3k

Public

uemacs

Random version of microemacs with my private modifications

● C ☆ 1.2k 🍴 233

Public

test-tlb

Stupid memory latency and TLB tester

● C ☆ 688 🍴 205

Public

pesconvert

Brother PES file converter

● C ☆ 395 🍴 69

Public

subsurface-for-dirk

Forked from subsurface/subsurface

Do not use - the real upstream is Subsurface-divelog/subsurface

● C++ ☆ 291 🍴 62

Public

libdc-for-dirk

Forked from subsurface/libdc

Only use for syncing with Dirk, don't use for anything else

● C ☆ 229 🍴 51

Public

2,705 contributions in the last year

Sep Oct Nov Dec Jan Feb Mar Apr May Jun Jul Aug Sep

Mon

Wed

Fri

Learn how we count contributions

Less More

@subsurface

2023

2022

2021

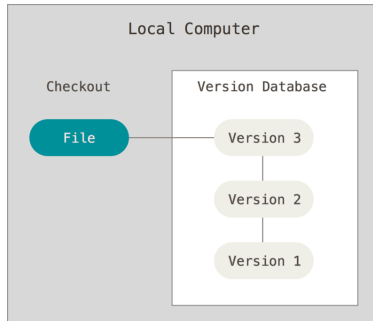
2020

2019

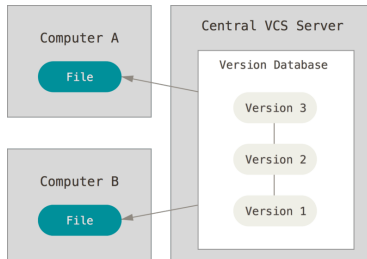
Types of version control: *local*

5/22

- Based on a simple database that keeps track of all the changes in the files
- Useful for solo projects
- No collaboration: **bad**



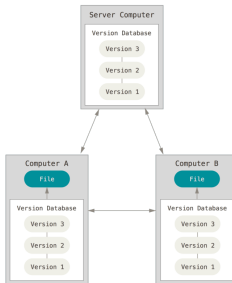
- A single server that contains all the versioned files
- Entire history of the project in a single place: **bad**

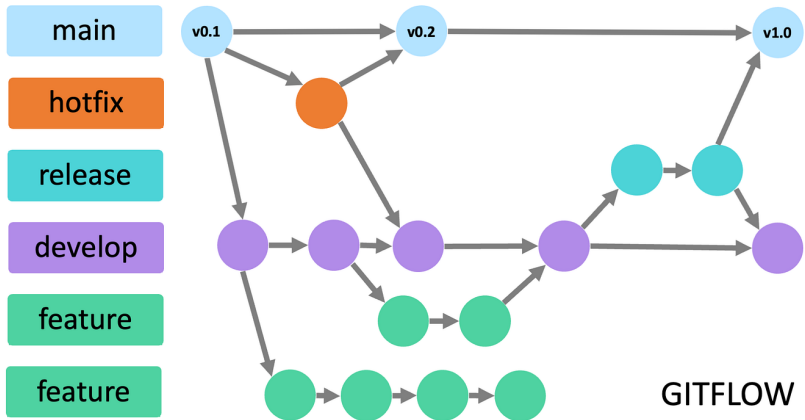


Types of version control: *distributed* (e.g. Git)

7/22

- Versioned files are stored at different places
- Any of the client repos can be copied back up to the server to restore it: **good**
- Every clone is a **full backup**, history included: you can commit, branch and inspect the log offline





Git in practice

Linux

```
# Debian / Ubuntu
sudo apt update
sudo apt install git
```

Windows

Download the installer from
<https://git-scm.com/download/win>

It ships with **Git Bash**, a Unix-like shell:
use that one.

Or use **WSL** in Windows: (`wsl --install`) and follow the Linux column.

```
# git --version
```

from the command line to check if the installation went through

- start a working area: `git init` | `git clone`
- mess around: `git add` | `git mv` | `git rm`
- examine history and state: `git status` | `git log` | `git diff`
- make and modify history: `git commit` | `git branch` | `git merge`
- collaborate: `git fetch` | `git pull` | `git push`

1. Working directory

what you see in your editor

`git add`

1 → 2

`git commit`

2 → 3

2. Staging area (index)what you *promised* to commit`git push`

3 → 4

`git fetch`

4 → 3

3. Local repository

your commits, on your disk

`git pull`

4 → 3 → 1

4. Remote repository

the shared copy (GitHub...)

Why it matters

Almost every confusing Git message is really the question “*which of these four places are we talking about?*”. `git status` answers it.

```
my-project/  
|-- .git/  
|-- .gitignore  
|-- README.md  
|-- src/  
|   |-- main.js  
|-- assets/
```

- `.git/` holds the entire history: delete it and the project becomes an ordinary folder
- `.gitignore` lists what Git must **not** track (`node_modules/`, build output, samples, `.env`)
- `README.md` says what the project is and how to run it

Both `.gitignore` and `README.md` belong in the very first commit.

- `git add`: stages changes (new, modified or deleted files) in the **staging area**, preparing them for a commit
- `git commit`: records the staged changes into the repo's history, creating a **snapshot of the current state** of the project

When to use what

The staging area exists so that you can **choose** what goes into a commit. Did you fix a bug *and* rename twelve variables? Stage them separately and make two commits. Future-you, reading the log, will be grateful.

- **git fetch**: retrieves updates from a remote repo **without altering your local working directory**
- **git pull**: retrieves updates **and then integrates** them into your current branch

The real difference

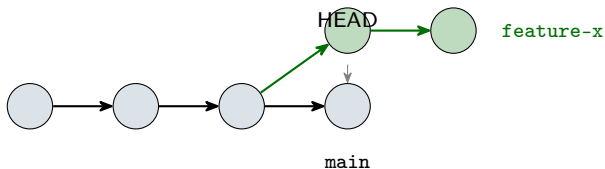
fetch is the **read-only, always-safe** move: it downloads the new commits but **your files do not change**. **pull** is fetch **plus** an operation that rewrites your working tree.

Almost synonyms across GitHub and GitLab.

A PR is a proposal to merge a set of changes from one branch into another, reviewed by team members

Note

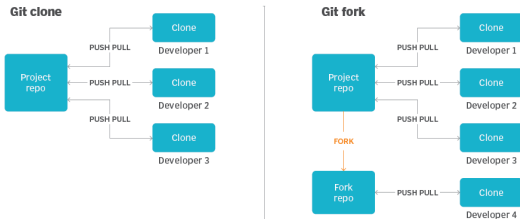
A PR is not a Git feature: Git only knows about `git merge`. The PR is the social layer built on top by GitHub/GitLab: discussion, review, CI checks, approvals, before the merge actually happens.



- `git switch -c feature-x`: create a new branch (pointer) and move onto it
- `git merge feature-x`: bring its commits into the current branch. The reading here is: "I'm in a specific branch, let's say the main, by calling this command I'm merging feature-x into the main branch"

Git clone vs. fork

Developers who work on a common codebase will clone the repository and then perform push and pull operations to synchronize their changes. In contrast, a fork creates a new codebase and updates to the fork are not synchronized with the original repo.



- `git checkout HEAD~3`: temporarily go back **without removing the latest commits** (“detached HEAD”)
- `git reset --soft HEAD~3`: permanently **reset the branch** to a previous commit (and keep changes staged)
- `git checkout -b new-branch HEAD~3`: create a new branch from a previous commit, **preserving the current branch**

HEAD = where you are now. HEAD~ 3 = three commits before that.

- **--soft**: moves the branch, changes stay **staged**
- **--mixed** (default): moves the branch, changes stay in the **working directory**, unstaged
"I committed too early, let me re-organise"
- **--hard**: moves the branch and **throws the changes away**
dangerous: uncommitted work is gone

```
git switch main && git pull      # start from the latest state
git switch -c fix/clipping-bug   # your own branch
... edit, test, repeat ...
git add -p
git commit -m "Clamp output buffer to [-1, 1]"
git push origin fix/clipping-bug
```

Then: open a pull request, get a review, merge, delete the branch.



Hands-on

1. Create a GitHub account
2. Populate a new repo with files/folders
3. Create a new branch, commit some changes, inspect the history
4. Pull requests and merge without conflicts
5. Like before, but with conflict resolution

1. Install Git
2. Create and initialize a local repo
3. Add and commit stuff
4. Tweak the history (the three methods above)
5. *Extra:* merge changes from new-branch into main (`git merge new-branch`)

1. Group in 2–4 people
2. Imagine a mockup project (app, library, book)
3. Assign tasks (devs, maintainers)
4. Create and clone the remote repo
5. Pull and push to branches (2–3 iterations)
6. Create and review pull requests for merging changes
7. Final goal: a clean, functional `main` branch